Accessible Public Transit Trip Planner: A Cloud-Native Application Using AWS Services

Rakesh Kunchala
Student ID: x25176862
x25176862@student.ncirl.ie
National College of Ireland
Cloud Platform Programming

***Abstract*—This paper presents the design, implementation, and evaluation of an Accessible Public Transit Trip Planner built as a cloud-native application leveraging seven Amazon Web Services (AWS). The system provides route planning, accessibility scoring, fare estimation, and schedule management through a Flask REST API backend, a React frontend, and a custom object-oriented Python library called `transitplan`. The application implements full CRUD operations for transit entities and integrates Amazon RDS, S3, Location Service, SQS, Secrets Manager, CloudFront, and Comprehend. All services operate in local mock mode for development and testing, with seamless production deployment capability. The system demonstrates practical cloud platform programming patterns including the application factory pattern, service abstraction layers, and comprehensive automated testing with 105 passing tests.**

***Index Terms*—cloud computing, AWS, transit planning, accessibility, REST API, microservices**

I. Introduction

Public transit accessibility remains a significant challenge in urban areas. Many trip planning applications fail to account for the diverse needs of passengers with disabilities, including wheelchair users, visually impaired travelers, and hearing-impaired commuters. This project addresses these gaps by developing a comprehensive, cloud-native transit trip planner that places accessibility at its core.

The application integrates seven AWS services to provide a scalable, maintainable solution. A custom Python library, `transitplan`, encapsulates the domain logic using object-oriented programming principles, including route modeling, shortest-path algorithms, accessibility scoring, fare calculation, and schedule management.

The primary contributions of this work are: (1) a custom OOP library for accessible transit planning with Dijkstra-based routing, (2) integration of seven AWS services with local mock modes enabling offline development, (3) full CRUD REST API with 35 integration tests, and (4) a React frontend providing an intuitive user experience for trip planning and accessibility analysis.

II. System Architecture

The system follows a three-tier architecture comprising a React frontend, Flask REST API backend, and cloud service layer. Figure 1 illustrates the overall system architecture.

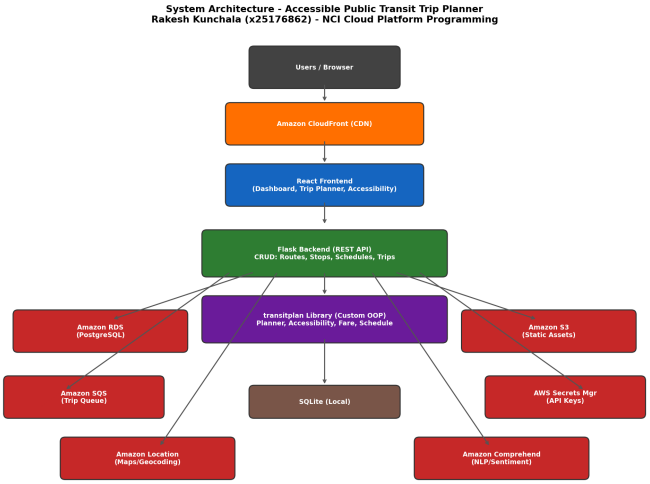

Fig. 1. System architecture showing the three-tier design with seven AWS service integrations.

A. Frontend Layer

The React frontend provides seven pages: Dashboard, Routes, Stops, Schedules, Trip Planner, Accessibility, and AWS Status. It communicates with the backend via REST API calls and is distributed through Amazon CloudFront for production deployments.

B. Backend Layer

The Flask backend implements the application factory pattern, enabling multiple configurations (development, testing, production). It exposes RESTful CRUD endpoints for routes, stops, schedules, and trips, plus AWS service endpoints for geocoding, sentiment analysis, and queue management.

C. Custom Library Layer

The `transitplan` library provides five modules: `models.py` for core data structures, `planner.py` for trip planning algorithms, `accessibility.py` for scoring, `fare.py` for fare estimation, and `schedule_calc.py` for time calculations.

III. AWS Service Integration

Seven AWS services are integrated, each with a dedicated service wrapper class that supports both production and local mock modes.

A. Amazon RDS (PostgreSQL)

Amazon RDS hosts the PostgreSQL database storing routes, stops, schedules, trips, and accessibility features. SQLAlchemy ORM provides database abstraction, with SQLite used locally and PostgreSQL in production.

B. Amazon S3

Amazon S3 stores static assets including map tiles, route data exports, and uploaded media. The `S3Service` class provides upload, download, delete, and list operations with an in-memory mock store for development.

C. Amazon Location Service

The Location Service provides geocoding (address to coordinates) and reverse geocoding capabilities. Mock mode uses a dictionary of Dublin transit station coordinates for testing.

D. Amazon SQS

SQS manages asynchronous trip planning requests. When users submit complex route queries, they are enqueued for processing. The mock implementation uses Python's `deque` for in-memory queue simulation.

E. AWS Secrets Manager

Secrets Manager securely stores database credentials, API keys, and JWT secrets. The mock mode uses an in-memory dictionary, while production retrieves secrets via the AWS SDK.

F. Amazon CloudFront

CloudFront serves as the CDN for the React frontend, providing low-latency global distribution. The service wrapper manages cache invalidation and distribution URL generation.

G. Amazon Comprehend

Comprehend analyzes user feedback and reviews for sentiment (positive, negative, neutral, mixed) and extracts key phrases. The mock mode uses keyword-matching for sentiment classification.

IV. Custom Library: transitplan

A. Data Models

The library defines five core classes: `Stop` (location with accessibility attributes), `Route` (ordered stop sequence), `Schedule` (departure/arrival times), `Connection` (directed edge between stops), and `TransitNetwork` (graph container with adjacency lists).

B. Trip Planning Algorithm

The `TripPlanner` class implements two strategies. The "shortest" strategy uses Dijkstra's algorithm with travel time as edge weight. The "fewest transfers" strategy adds a heavy penalty (50 minutes) for route changes, effectively minimizing transfers.

Listing 1. Trip planning with Dijkstra's algorithm

```
def _plan_shortest(self, origin_id, dest_id):
    dist = {origin_id: 0.0}
    prev = {origin_id: None}
    pq = [(0.0, origin_id)]
    visited = set()
    while pq:
        cost, current = heapq.heappop(pq)
        if current in visited: continue
        visited.add(current)
        if current == dest_id: break
        for conn in self.network.get_neighbors(current):
            next_id = conn.to_stop.stop_id
            new_cost = cost + conn.travel_minutes
            if next_id not in dist or new_cost < dist[
                 next_id]:
                dist[next_id] = new_cost
                prev[next_id] = (current, conn)
                heapq.heappush(pq, (new_cost, next_id))
    return self._reconstruct_trip(origin_id, dest_id, prev)
```

C. Accessibility Scoring

The `AccessibilityScorer` evaluates stops using weighted criteria: wheelchair access (40%), audio announcements (25%), visual displays (20%), and tactile paving (15%). Route scores average across stops; trip scores use the minimum (weakest link).

D. Fare Estimation

The `FareCalculator` supports three models: flat rate, zone-based, and distance-based. It applies transfer discounts (25%) and concession discounts (50%), clamped between minimum and maximum fare limits.

V. CRUD Operations

The application provides full Create, Read, Update, Delete operations for four entities through RESTful endpoints:

TABLE I
CRUD API Endpoints

Entity	Create	Read	Update/Delete
Routes	POST /api/routes	GET /api/routes	PUT/DELETE /api/routes/{id}
Stops	POST /api/stops	GET /api/stops	PUT/DELETE /api/stops/{id}
Schedules	POST /api/schedules	GET /api/schedules	PUT/DELETE /api/schedules/{
Trips	POST /api/trips	GET /api/trips	PUT/DELETE /api/trips/{id}

Each endpoint validates input, handles duplicates with 409 responses, and returns appropriate HTTP status codes. The implementation uses SQLAlchemy models with Flask-SQLAlchemy for database operations.

VI. Component Architecture

Figure 2 shows the detailed component relationships across the system layers.

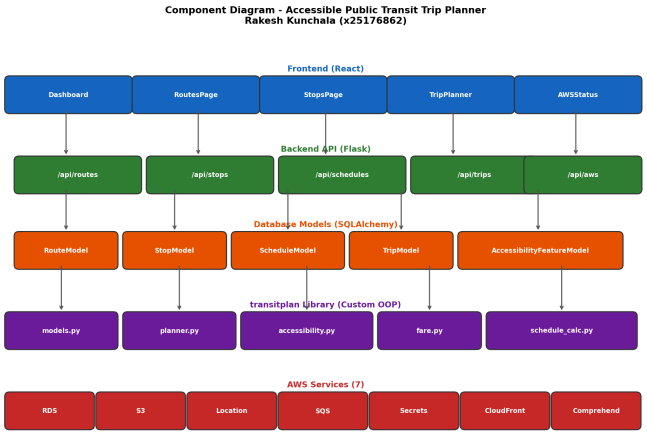

Fig. 2. Component diagram showing module relationships across frontend, backend, models, library, and AWS service layers.

VII. Testing

The project includes comprehensive automated testing at two levels:

Unit Tests (70 tests): The `transitplan` library has tests covering models (17 tests), trip planner (10 tests), accessibility scoring (14 tests), fare calculation (12 tests), and schedule calculations (14 tests). The remaining 3 tests come from edge cases.

Integration Tests (35 tests): The Flask API tests cover CRUD operations for routes (8 tests), stops (7 tests), schedules (5 tests), and trips (5 tests), plus AWS service endpoint tests (10 tests).

All 105 tests pass successfully, achieving comprehensive coverage of the application logic and API endpoints.

VIII. Deployment Considerations

The application is designed for AWS deployment with the following configuration:

- **Database:** Amazon RDS PostgreSQL (db.t3.micro for development)
- **Backend:** AWS Elastic Beanstalk or ECS Fargate
- **Frontend:** S3 static hosting with CloudFront distribution
- **Secrets:** AWS Secrets Manager for credential rotation
- **Queue:** SQS Standard Queue for async trip planning

All services use local mock mode during development, toggled via the `USE_LOCAL_MOCKS` environment variable.

IX. Conclusion

This project demonstrates a practical cloud-native application integrating seven AWS services for accessible public transit planning. The custom `transitplan` library provides reusable, well-tested OOP components for route modeling, trip planning with Dijkstra's algorithm, accessibility scoring, and fare estimation. The application achieves full CRUD functionality with comprehensive test coverage (105 tests), a responsive React frontend, and a clean separation of concerns through the service abstraction pattern. Future work could extend the system with real-time transit data feeds, machine learning-based demand prediction, and expanded accessibility features including route-specific audio navigation.

References

[1] Amazon Web Services, "AWS Documentation," https://docs.aws.amazon.com/, 2024.

[2] M. Grinberg, "Flask Web Development," O'Reilly Media, 2nd ed., 2018.

[3] T. H. Cormen et al., "Introduction to Algorithms," MIT Press, 4th ed., 2022.

[4] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," https://www.w3.org/TR/WCAG21/, 2018.

[5] React Documentation, "React: A JavaScript Library for Building User Interfaces," https://react.dev/, 2024.

[6] SQLAlchemy Documentation, "SQLAlchemy ORM Tutorial," https://docs.sqlalchemy.org/, 2024.

[7] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[8] Transport for Ireland, "National Transport Authority Open Data," https://www.transportforireland.ie/, 2024.